

TechTribers

Java Interview Series · Part 10 · Q398–Q427

Modern Java 22 → 27

Unnamed variables, flexible constructors, gatherers, the FFM and Class-File APIs, the AOT cache, compact headers, G1 everywhere, post-quantum crypto, previews to watch and Project Valhalla, verified on JDK 25, 26 and 27.

30	25	15	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `MJ-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

30	5	25	15
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

✨	01	Language Features, Java 22–25	Q398–Q402	4
🧰	02	Libraries & Tooling, Java 22–26	Q403–Q412	9
🚀	03	Runtime, GC & Start-up, Java 24–27	Q413–Q418	18
🔭	04	Previews & Incubators in JDK 27	Q419–Q422	25
🎓	05	Java 25 LTS, 26 & 27, and Choosing a Version	Q423–Q427	30

© TechTribers. For personal use only. Do not redistribute.

All questions

✨ Language Features, Java 22–25

Q398 What are unnamed variables and patterns (_) in Java 22? 4

Q399 What are flexible constructor bodies (Java 25)? 5

Q400 What are compact source files and instance main methods (Java 25)? 6

Q401 What are module import declarations (Java 25)? 7

Q402 Why were string templates removed from Java? 8

🧰 Libraries & Tooling, Java 22–26

Q403 What is the Foreign Function & Memory (FFM) API, and how does it replace JNI? 9

Q404 What are Stream Gatherers (Java 24)? 10

Q405 What is the Class-File API (Java 24)? 11

Q406 How does the multi-file source-code launcher work (Java 22)? 12

Q407 What are Markdown documentation comments (Java 23)? 13

Q408 How does HTTP/3 work in Java's HttpClient (Java 26)? 14

Q409 What is the Key Derivation Function (KDF) API (Java 25)? 15

Q410 What post-quantum cryptography does Java support (ML-KEM, ML-DSA, hybrid TLS)? 15

Q411 What are the PEM encodings of cryptographic objects (preview, final planned in JDK 28)? 17

Q412 What JDK Flight Recorder improvements arrived in Java 25–27? 17

🚀 Runtime, GC & Start-up, Java 24–27

Q413 What is the AOT cache (Project Leyden), and how do you use it? 18

Q414 Compact headers are the default in JDK 27: how much heap do they really save, and how do you roll them out? 20

Q415 What changed for garbage collectors between Java 22 and 27, and why is G1 now the default everywhere? 21

Q416 What is "integrity by default", and which changes trigger new warnings after an upgrade? 22

Q417 What does JEP 500, "Prepare to Make Final Mean Final", change? 23

Q418 What was removed or disabled between Java 22 and 27? 24

🔭 Previews & Incubators in JDK 27

Q419 What are primitive types in patterns, `instanceof` and `switch`? 25

Q420 What are Lazy Constants (formerly Stable Values)? 26

Q421 Which features are still preview or incubating in JDK 27, and when will they be final? 27

Q422 What is Project Valhalla, and what is coming in JDK 28? 28

🎓 Java 25 LTS, 26 & 27, and Choosing a Version

Q423 What's new in Java 25, the current LTS? 30

Q424 What did Java 26 and Java 27 add? 31

Q425 How do you upgrade a service from Java 21 to 25, and what are the gains? 32

Q426 Which Java version should you run in production in 2026? 33

Q427 "What has changed in Java since Java 8?" Give a strong 60-second answer. 34

© TechTribers. For personal use only. Do not redistribute.

Section 01

Language Features, Java 22–25

The small language changes that remove boilerplate: unnamed variables, constructors that can validate first, scripts without a class, and one feature that was withdrawn.

Questions 398–402 · 5 questions



Q398 MJ-059

• Intermediate

What are unnamed variables and patterns (_) in Java 22?

⚡ Quick answer

The underscore `_` declares a variable or pattern you intentionally never use: a loop variable, a caught exception, a lambda parameter, a record component, or a pattern in `switch`. It was a preview in Java 21 and is final in Java 22 (JEP 456).

You can use `_` several times in the same scope, and it makes the intent explicit: readers and static-analysis tools know the value is deliberately ignored.

Java

```
sealed interface Shape permits Circle, Sq { }
record Circle(double r) implements Shape { }
record Sq(double s) implements Shape { }
record Point(int x, int y) { }

int total = 0;
for (var _ : List.of("a", "b", "c")) total++;                    // loop variable never used
System.out.println(total);

try { Integer.parseInt("x"); }
catch (NumberFormatException _) { System.out.println("bad number"); }   // exception never used

Object o = new Point(3, 4);
if (o instanceof Point(int x, _)) System.out.println("x=" + x);    // ignore the y component

Shape s = new Circle(2);
String kind = switch (s) { case Circle _ -> "circle"; case Sq _ -> "square"; };   // no binding
needed
System.out.println(kind);

var counts = new HashMap<String, Integer>();
counts.computeIfAbsent("k", _ -> 1);                             // unused lambda parameter
System.out.println(counts);
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
3
bad number
x=3
circle
{k=1}
```

⚠️ **Interview trap**

`_` can't be *read*. Using it as an expression is a compile error, and it still can't be a field or method name. Old code that used `_` as an identifier stopped compiling in Java 9 and only got the new meaning in Java 22.

Q399 MJ-060 • Advanced

What are flexible constructor bodies (Java 25)?

⚡ **Quick answer**

Statements may now appear **before** `super(...)` or `this(...)` in a constructor, as long as they don't use the object under construction (`this`). Validation, argument preparation and field initialisation can happen first. Final in Java 25 (JEP 513) after previews in 22, 23 and 24.

Before Java 25 you had to hide this logic in static helper methods (`super(normalize(name))`) or validate *after* the superclass constructor had already run. The rule now: the pre- `super` section can't read `this` , call instance methods, or use instance fields (assigning the class's own fields is allowed), because the object isn't built yet.

A bonus fix: if a superclass constructor calls an overridden method, the subclass can now set its own fields first, so the override no longer sees default values.

Java

```java
static class Base {
    final String name;
    Base(String n) { name = n; System.out.println("Base(" + n + ")"); }
}
static class User extends Base {
    final int age;
    User(String name, int age) {
        if (age < 0) throw new IllegalArgumentException("age < 0");     // validate BEFORE super
        String normalized = name.strip().toLowerCase();                 // prepare the argument
        super(normalized);
        this.age = age;
    }
}
var u = new User("  ASHA ", 30);
System.out.println(u.name + " " + u.age);
try { new User("x", -1); } catch (IllegalArgumentException e) { System.out.println(e.getMessage());
}
```

▶ **Output**

```
Base(asha)
asha 30
```


© TechTribers. For personal use only. Do not redistribute.

```
age < 0
```

💡 **Pro tip**

This removes a long-standing reason people reached for factory methods or builders just to validate arguments before calling a parent constructor.

📜 **Q400** MJ-061 • Basic

What are compact source files and instance main methods (Java 25)?

⚡ **Quick answer**

A source file can be just a `void main()` method: no class, no `public static`, no `String[] args`. The compiler wraps it in an implicit class, imports `java.base` automatically, and `IO.println` replaces `System.out.println`. Final in Java 25 (JEP 512); it exists to make small programs and teaching simpler.

Run it with `java Hello.java` (no separate compile step). The `IO` class is `java.lang.IO` with `print`, `println` and `readln` helpers for console I/O.

The launcher looks for a main method in this order: `static void main(String[])`, `static void main()`, instance `void main(String[])`, instance `void main()`. A compact file may also declare fields and other methods, and it grows into a normal class when you add a `class` declaration.

Java

```java
void main() {
    var langs = List.of("Java", "Kotlin", "Scala");          // List needs no import: java.base is implicit
    for (String l : langs) IO.println("hello " + l);
    IO.println(langs.stream().map(String::length).toList());
}
```

▶ **Output**

```
hello Java
hello Kotlin
hello Scala
[4, 6, 5]
```

💡 **Pro tip**

Great for scripts, interview whiteboard snippets and teaching. Production code still lives in normal classes inside packages built by Maven or Gradle.

© TechTribers. For personal use only. Do not redistribute.

Q401 MJ-062

• Intermediate

What are module import declarations (Java 25)?

⚡ **Quick answer**

`import module java.base;` imports every public type in every package exported by that module (and the modules it transitively requires) with one line. Final in Java 25 (JEP 511). It is a convenience for scripts and learning, not a recommendation for large codebases.

Facts worth knowing:

- It works with named and unnamed modules; for your own modules, `import module com.tt.billing;` works the same way.
- Ambiguities (for example `java.util.Date` vs `java.sql.Date` after `import module java.se;`) are resolved with an explicit single-type import.
- Compact source files already import `java.base` implicitly.

Java

```
import module java.base;

public class Main {
    public static void main(String[] args) {
        List<String> l = new ArrayList<>(List.of("b", "a"));          // java.util
        Collections.sort(l);
        System.out.println(l + " " + Stream.of(1, 2, 3).map(x -> x * 2).toList()    //
java.util.stream
                + " " + LocalDate.of(2026, 10, 14).getDayOfWeek());                 // java.time
    }
}
```

▶ **Output**

```
[a, b] [2, 4, 6] WEDNESDAY
```

⚠️ **Interview trap**

Wildcard-style imports hide where a class comes from and can produce surprising name clashes. Most teams keep explicit imports in production code and use module imports for scratch files.

© TechTribers. For personal use only. Do not redistribute.

Q402 MJ-063

• Intermediate

Why were string templates removed from Java?

⚡ Quick answer

String templates (`STR."Hello \{name}"`) were previewed in Java 21 and 22, then **removed in Java 23**: the design of template processors needed to be rethought, so the preview was withdrawn rather than finalised. Code using the old syntax does not compile on current JDKs. A redesigned version may return later.

What to use today: `String.formatted` , `String.format` , `MessageFormat` , text blocks, `StringBuilder` , or a templating library for HTML/SQL/JSON. Safe SQL always means `PreparedStatement` parameters, whatever the string syntax.

Why this question is asked: it checks whether you understand the **preview** model. A preview feature can change or disappear entirely, which is exactly why production code shouldn't depend on one.

Java

```
String name = "Asha"; int orders = 3; double total = 1499.5;

String a = "Hello %s, you have %d orders worth INR %,.2f".formatted(name, orders, total);
String b = MessageFormat.format("Hello {0}, you have {1} orders", name, orders);
String c = """
        Hello %s,
        orders: %d
        """.formatted(name, orders);                    // text block + formatted()
System.out.println(a);
System.out.println(b);
System.out.print(c);
```

▶ Output

```
Hello Asha, you have 3 orders worth INR 1,499.50
Hello Asha, you have 3 orders
Hello Asha,
orders: 3
```

⚠️ Interview trap

Tutorials from 2023–24 still show the `STR.` syntax as if it were Java 21+ standard. It never was final, and it no longer compiles.

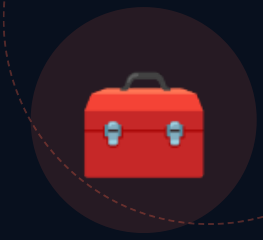
© TechTribers. For personal use only. Do not redistribute.

Section 02

Libraries & Tooling, Java 22–26

Calling native code safely, new stream operations, reading and writing class files, running multi-file programs, HTTP/3, and post-quantum cryptography.

Questions 403–412 · 10 questions



Q403 MJ-064

• Advanced

What is the Foreign Function & Memory (FFM) API, and how does it replace JNI?

⚡ Quick answer

FFM (final in Java 22, JEP 454) lets Java call C functions and manage native memory with plain Java code: a `Linker` creates a `MethodHandle` for a native function, and an `Arena` owns off-heap `MemorySegment`s with explicit, checked lifetimes. No C glue code, no JNI headers.

Safety features JNI never had:

- **Spatial safety**: reading past a segment's size throws `IndexOutOfBoundsException` instead of corrupting memory.
- **Temporal safety**: using a segment after its arena closed throws `IllegalStateException` ("Already closed").
- **Explicit lifetimes**: `Arena.ofConfined()` (one thread), `ofShared()`, `ofAuto()` (GC-managed), `global()` (never freed).

The `jextract` tool generates Java bindings from C header files, so you rarely write `FunctionDescriptor`s by hand.

Java

```
import java.lang.foreign.*;
import java.lang.invoke.*;

public class Main {
    public static void main(String[] a) throws Throwable {
        Linker linker = Linker.nativeLinker();
        MethodHandle strlen = linker.downcallHandle(
                linker.defaultLookup().find("strlen").orElseThrow(),
                FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));

        try (Arena arena = Arena.ofConfined()) {
            MemorySegment s = arena.allocateFrom("TechTribers");      // C string in native memory
            long len = (long) strlen.invokeExact(s);                  // call libc strlen
            System.out.println("strlen = " + len);

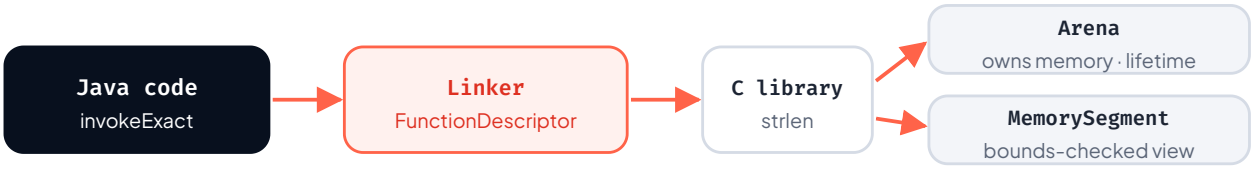
            MemorySegment seg = arena.allocate(ValueLayout.JAVA_INT, 4);
            for (int i = 0; i < 4; i++) seg.setAtIndex(ValueLayout.JAVA_INT, i, (i + 1) * 10);
```


© TechTribers. For personal use only. Do not redistribute.

```
            int sum = 0;
            for (int i = 0; i < 4; i++) sum += seg.getAtIndex(ValueLayout.JAVA_INT, i);
            System.out.println("sum = " + sum + ", byteSize = " + seg.byteSize());
        }                                                    // memory freed here,
deterministically
    }
}
```

▶ Output

```
strlen = 11
sum = 100, byteSize = 16
```



Arena.ofConfined() one thread · ofShared() many threads · ofAuto() GC-managed · global() never freed

Safe by design: segments are bounds-checked and time-checked (no use-after-free); native access needs --enable-native-access. jextract generates Java bindings from C headers, replacing hand-written JNI.

⚠️ **Interview trap**

Native access is a restricted operation. Without `--enable-native-access=ALL-UNNAMED` (or a module name) newer JDKs print a warning, and a future release will make it an error.

Q404 MJ-065 • Advanced

What are Stream Gatherers (Java 24)?

⚡ **Quick answer**

Gatherers (final in Java 24, JEP 485) are custom **intermediate** stream operations. `Stream.gather(...)` accepts built-ins such as `windowFixed`, `windowSliding`, `scan`, `fold` and `mapConcurrent`, or your own `Gatherer`, filling the gap where `map` / `filter` / `flatMap` weren't enough.

`mapConcurrent` runs each mapping on its own virtual thread with a concurrency limit, a neat way to call a remote service for many items without managing an executor. The existing terminal-only limit (`Collector`) is mirrored by gatherers on the intermediate side.

Java

```
System.out.println(Stream.of(1, 2, 3, 4, 5, 6, 7).gather(Gatherers.windowFixed(3)).toList());
System.out.println(Stream.of(1, 2, 3, 4, 5).gather(Gatherers.windowSliding(2)).toList());
System.out.println(Stream.of(1, 2, 3, 4).gather(Gatherers.scan(() -> 0, (acc, x) -> acc +
x)).toList());   // running total
System.out.println(Stream.of(1, 2, 3, 4).gather(Gatherers.fold(() -> 0, (acc, x) -> acc +
x)).toList());   // one result
```

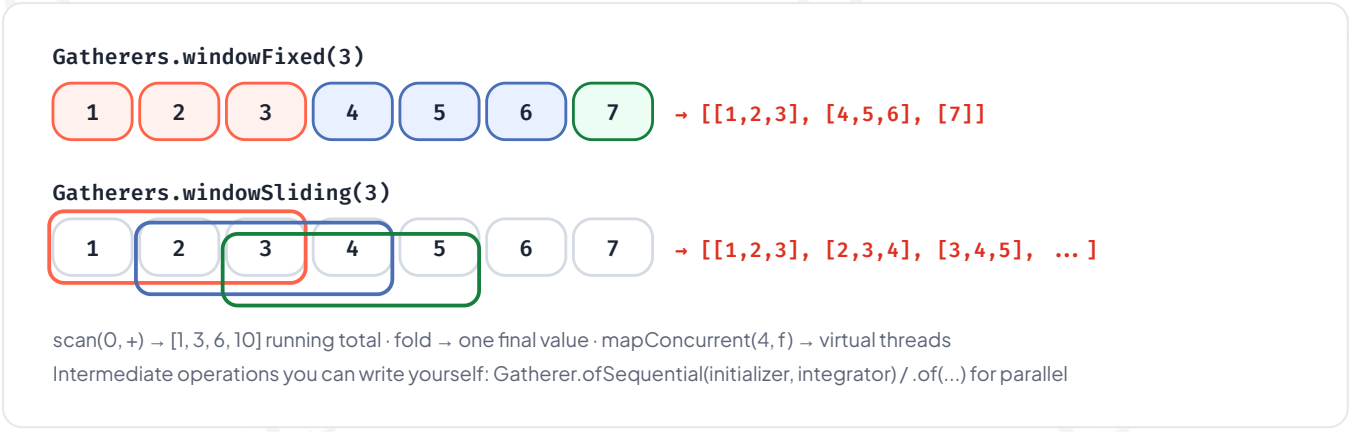

© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(Stream.of("a", "bb", "ccc").gather(Gatherers.mapConcurrent(4,
String::length)).toList());

// A custom gatherer: drop consecutive duplicates
Gatherer<Integer, ?, Integer> distinctConsecutive = Gatherer.ofSequential(
        () -> new int[]{Integer.MIN_VALUE, 0},
        (state, x, downstream) -> {
            if (state[1] == 0 || state[0] != x) { state[0] = x; state[1] = 1; return
downstream.push(x); }
            return true;
        });
System.out.println(Stream.of(1, 1, 2, 2, 2, 3, 1, 1).gather(distinctConsecutive).toList());
```

▶ Output

```
[[1, 2, 3], [4, 5, 6], [7]]
[[1, 2], [2, 3], [3, 4], [4, 5]]
[1, 3, 6, 10]
[10]
[1, 2, 3]
[1, 2, 3, 1]
```



scan(0, +) → [1, 3, 6, 10] running total · fold → one final value · mapConcurrent(4, f) → virtual threads
Intermediate operations you can write yourself: Gatherer.ofSequential(initializer, integrator) / .of(...) for parallel

💡 **Pro tip**

Typical uses: batching records before a bulk insert (`windowFixed`), moving averages (`windowSliding`), running totals (`scan`), and de-duplicating adjacent events.

🔬 **Q405** MJ-066 • Advanced

What is the Class-File API (Java 24)?

⚡ **Quick answer**

`java.lang.classfile` (final in Java 24, JEP 484) is the standard API to **parse, generate and transform** `.class` files. The JDK uses it internally instead of a private copy of ASM, and frameworks can migrate from ASM/Byte Buddy so they no longer lag behind new Java versions.

Why it matters:

© TechTribers. For personal use only. Do not redistribute.

- Java releases every six months and the class-file format changes with each one. Libraries bundling ASM had to wait for a compatible ASM release before supporting a new JDK; a JDK-owned API removes that lag.
- Immutable, fluent API with builders, instead of ASM's visitor callbacks.

```java
import java.lang.classfile.*;
import java.lang.constant.*;
import java.lang.invoke.*;
import static java.lang.constant.ConstantDescs.*;
import static java.lang.classfile.ClassFile.*;

// 1. Parse an existing class
ClassModel cm =
ClassFile.of().parse(Integer.class.getResourceAsStream("Integer.class").readAllBytes());
long overloads = cm.methods().stream().filter(m -> m.methodName().equalsString("parseInt")).count();
System.out.println("Integer has " + overloads + " parseInt overloads, super=" +
cm.superclass().get().asInternalName());

// 2. Generate a class at runtime: public static int answer() { return 42; }
byte[] bytes = ClassFile.of().build(ClassDesc.of("GeneratedAnswer"), cb -> cb
        .withMethodBody("answer", MethodTypeDesc.of(CD_int), ACC_PUBLIC | ACC_STATIC, code ->
code.bipush(42).ireturn()));
Class<?> c = MethodHandles.lookup().defineClass(bytes);
System.out.println(c.getName() + ".answer() = " + c.getMethod("answer").invoke(null));
```

▶ Output

```
Integer has 3 parseInt overloads, super=java/lang/Number
GeneratedAnswer.answer() = 42
```

💡 **Pro tip**

You'll rarely use it directly. Know it as the reason Spring, Mockito, Hibernate and Byte Buddy upgrades keep appearing right after each JDK release.

Q406 MJ-067 • Basic

How does the multi-file source-code launcher work (Java 22)?

⚡ **Quick answer**

Since Java 22 (JEP 458), `java Main.java` compiles and runs a program whose classes are spread over several source files, as long as the other files sit next to it (or in the matching package directories). Before Java 22, single-file launch failed with "cannot find symbol" as soon as a second file was involved.

The classes are compiled in memory and nothing is written to disk. You can also pass `--class-path` for library JARs. Useful for scripts, demos, bug reports ("here's a 3-file reproducer") and teaching, while real projects still need Maven or Gradle.

© TechTribers. For personal use only. Do not redistribute.

Java

```java
// Main.java
public class Main {
    public static void main(String[] a) { System.out.println(Helper.greet("TechTribers") + " " +
Util.twice(21)); }
}

// Helper.java
class Helper { static String greet(String n) { return "Hello, " + n; } }

// Util.java
class Util { static int twice(int x) { return x * 2; } }

// $ java Main.java        (no javac, no build tool)
// Hello, TechTribers 42
```

▶ Output

```
Hello, TechTribers 42
```

⚠️ **Interview trap**

Source-launcher mode compiles with default options and no incremental builds. A change in one file recompiles everything on each run, which is fine for scripts and slow for anything big.

📝 **Q407** MJ-068 • Basic

What are Markdown documentation comments (Java 23)?

⚡ **Quick answer**

JEP 467 lets Javadoc comments be written in Markdown: start each line with `///` instead of wrapping `/* ... /`. You get headings, lists, code fences, tables and links without HTML tags like `<p>` and `<pre>`.

Existing `/* /` comments keep working, and block tags (`@param`, `@return`, `@throws`) are the same. In Markdown comments, `[Order]` is a reference link to a type.

Java

```java
/// Calculates the delivery date for an order.
///
/// Rules:
/// - Orders before 14:00 ship the same day
/// - Sundays and holidays are skipped
///
/// ```java
/// LocalDate d = DeliveryCalendar.estimate(order);
/// ```
///
/// @param order the order to ship
/// @return the estimated delivery date
```


© TechTribers. For personal use only. Do not redistribute.

```
/// @throws IllegalArgumentException if the order has no items
public LocalDate estimate(Order order) { ... }

/// A link to another type: [Order] or a method: [Order#total()].
```

💡 **Pro tip**

Pair this with Javadoc snippets (Java 18's `{@snippet ...}`) so the examples in your API docs are compiled and tested along with your code.

🌐 **Q408** MJ-069 • Intermediate

How does HTTP/3 work in Java's HttpClient (Java 26)?

⚡ **Quick answer**

Java 26 (JEP 517) adds HTTP/3, which runs over QUIC (UDP) instead of TCP, to `java.net.http.HttpClient`. It is **opt-in**: request `Version.HTTP_3` on the client or the request, and the client falls back to HTTP/2 or HTTP/1.1 if the server doesn't support HTTP/3.

Why HTTP/3: QUIC removes TCP head-of-line blocking across streams, sets up connections faster (combined transport and TLS 1.3 handshake) and survives network changes such as switching from Wi-Fi to mobile.

Practical points: the server must support HTTP/3 (many CDNs do), UDP port 443 has to be reachable (some corporate firewalls block it), and the fallback means your code works either way.

Java

```
HttpClient client = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_3)             // prefer HTTP/3, fall back if unavailable
        .connectTimeout(Duration.ofSeconds(3))
        .build();

HttpRequest request = HttpRequest.newBuilder(URI.create("https://example.com/"))
        .version(HttpClient.Version.HTTP_3)             // can also be requested per request
        .timeout(Duration.ofSeconds(5))
        .build();

System.out.println(client.version() + " " + request.version().get());
```

▶ **Output**

```
HTTP_3 HTTP_3
```

💡 **Pro tip**

Still always set connect and request timeouts. Which protocol was actually used is available on the response with `response.version()`.

© TechTribers. For personal use only. Do not redistribute.

Q409 MJ-070

• Advanced

What is the Key Derivation Function (KDF) API (Java 25)?

⚡ Quick answer

`javax.crypto.KDF` (final in Java 25, JEP 510) is a standard API for deriving keys from a secret, for example with HKDF as used in TLS 1.3. It replaces ad-hoc code that built HKDF out of `Mac` by hand.

HKDF has two steps, **extract** (concentrate the entropy of the input secret using a salt) and **expand** (stretch it into one or more keys, each bound to an `info` label). Deriving separate keys for encryption and for authentication from one secret is the standard use.

Java

```
import javax.crypto.spec.HKDFParameterSpec;
import java.security.spec.AlgorithmParameterSpec;

KDF hkdf = KDF.getInstance("HKDF-SHA256");
byte[] ikm = "master-secret".getBytes();        // input keying material
byte[] salt = "salt".getBytes();
byte[] info = "tt-session".getBytes();          // context: binds the key to its purpose

AlgorithmParameterSpec spec =
HKDFParameterSpec.ofExtract().addIKM(ikm).addSalt(salt).thenExpand(info, 32);
SecretKey k1 = hkdf.deriveKey("AES", spec);
SecretKey k2 = hkdf.deriveKey("AES", spec);
System.out.println(k1.getAlgorithm() + " " + k1.getEncoded().length + " bytes, deterministic=" +
Arrays.equals(k1.getEncoded(), k2.getEncoded()));
```

▶ Output

```
AES 32 bytes, deterministic=true
```

⚠️ Interview trap

A KDF is for high-entropy secrets (a key-exchange result). To store user passwords use a *password hashing* scheme such as Argon2, bcrypt or PBKDF2 (Spring Security's `PasswordEncoder`), never HKDF.

Q410 MJ-071

• Advanced

What post-quantum cryptography does Java support (ML-KEM, ML-DSA, hybrid TLS)?

⚡ Quick answer

Java 24 added the NIST-standard quantum-resistant algorithms **ML-KEM** (key encapsulation, JEP 496) and **ML-DSA** (signatures, JEP 497); JDK 27 adds **hybrid post-quantum key exchange for TLS 1.3** (JEP 527). They protect against "harvest now, decrypt later" attacks by a future quantum computer.

© TechTribers. For personal use only. Do not redistribute.

Notice the sizes: an ML-KEM-768 ciphertext is 1,088 bytes and an ML-DSA-65 signature is 3,309 bytes, far larger than their elliptic-curve equivalents (tens of bytes). That is the main cost of post-quantum algorithms, and why TLS uses **hybrid** key exchange (a classical X25519 exchange *and* ML-KEM in one handshake): you stay secure if either one is broken.

For most applications the practical step is upgrading the JDK and keeping TLS libraries current; Java handles the hybrid exchange inside the TLS stack.

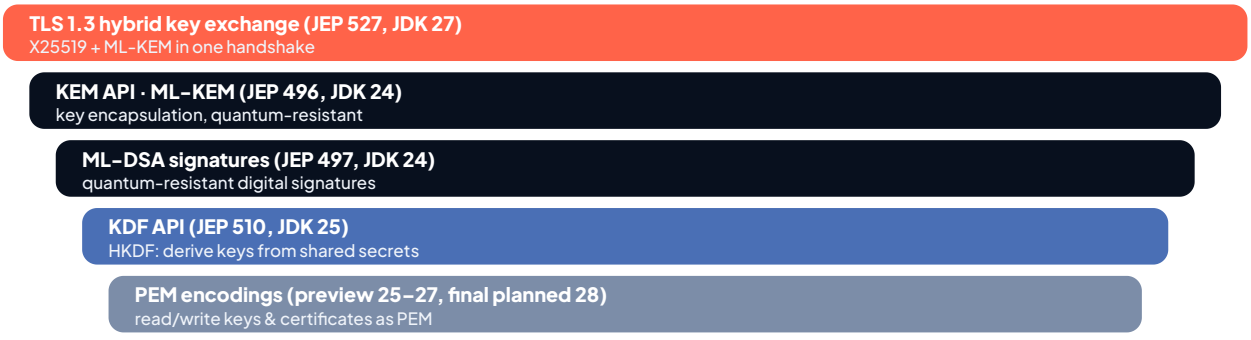
Java

```
KeyPairGenerator g = KeyPairGenerator.getInstance("ML-KEM-768");
KeyPair kp = g.generateKeyPair();
KEM kem = KEM.getInstance("ML-KEM");
KEM.Encapsulated enc = kem.newEncapsulator(kp.getPublic()).encapsulate();          // sender
SecretKey recovered = kem.newDecapsulator(kp.getPrivate()).decapsulate(enc.encapsulation());   //
receiver
System.out.println("ML-KEM secrets match: " + Arrays.equals(enc.key().getEncoded(),
recovered.getEncoded())
        + ", ciphertext bytes=" + enc.encapsulation().length + ", secret bytes=" +
recovered.getEncoded().length);

KeyPair sk = KeyPairGenerator.getInstance("ML-DSA-65").generateKeyPair();
Signature s = Signature.getInstance("ML-DSA");
byte[] msg = "invoice-42".getBytes();
s.initSign(sk.getPrivate()); s.update(msg); byte[] sig = s.sign();
s.initVerify(sk.getPublic()); s.update(msg);
System.out.println("ML-DSA verify: " + s.verify(sig) + ", signature bytes=" + sig.length);
```

▶ Output

```
ML-KEM secrets match: true, ciphertext bytes=1088, secret bytes=32
ML-DSA verify: true, signature bytes=3309
```



Goal: protect today's traffic against 'harvest now, decrypt later' attacks by quantum computers.

⚠ **Interview trap**

Quantum-resistant signatures are not a drop-in for JWT or certificate chains yet: ecosystem support (CAs, browsers, JWT libraries) is still catching up. Key *exchange* is the urgent part, because recorded traffic can be decrypted later.

© TechTribers. For personal use only. Do not redistribute.

Q411 MJ-072

• Intermediate

What are the PEM encodings of cryptographic objects (preview, final planned in JDK 28)?

⚡ Quick answer

A new API (`PEMEncoder` and `PEMDecoder`) converts keys, certificates and CRLs to and from the PEM text format (`-----BEGIN ...-----`), replacing the usual hand-rolled Base64 and header code. It has been in preview since Java 25 (third preview in JDK 27); JEP 542 plans to finalise it in JDK 28.

The third preview (JDK 27) renamed the `DEREncodable` interface to `BinaryEncodable` and made the `PEM` record a regular class, so earlier-preview code needs small edits. PEM files are what you meet in practice with `openssl`, Kubernetes secrets, TLS certificates and SSH keys.

Java

```
KeyPair kp = KeyPairGenerator.getInstance("Ed25519").generateKeyPair();

String pem = PEMEncoder.of().encodeToString(kp.getPrivate());
System.out.println(pem.lines().findFirst().get() + " ... " + pem.lines().reduce((x, y) -> y).get());

PrivateKey back = PEMDecoder.of().decode(pem, PrivateKey.class);
System.out.println("round trip equal: " + back.equals(kp.getPrivate()));

String pub = PEMEncoder.of().encodeToString(kp.getPublic());
System.out.println(pub.lines().findFirst().get());
```

▶ Output

```
-----BEGIN PRIVATE KEY----- ... -----END PRIVATE KEY-----
round trip equal: true
-----BEGIN PUBLIC KEY-----
```

⚠️ Interview trap

Needs `--enable-preview` on the exact JDK (here 27). Until it's final, keep it out of libraries and use it only in tools or experiments.

Q412 MJ-073

• Advanced

What JDK Flight Recorder improvements arrived in Java 25–27?

⚡ Quick answer

JFR got sharper profiling and safer sharing: CPU-time profiling that samples from OS timers (JEP 509, experimental, Linux), cooperative sampling for a safer stack walker (JEP 518), method timing and tracing events (JEP 520), and in-process redaction of sensitive data (JEP 536, JDK 27).

How the CPU-time sampler differs from the older execution sampler: it is driven by the kernel's CPU timers, so it sees time spent in native code and doesn't depend on threads reaching a safepoint. In a quick test on JDK 25 a

© TechTribers. For personal use only. Do not redistribute.

1.5-second busy loop produced many `jdk.CPUTimeSample` events with a 10 ms throttle.

`jdk.MethodTiming` and `jdk.MethodTrace` record how long specific methods take without editing code or adding an agent, which fits hot-spot hunting in production.

Terminal

```
# JDK 25 on Linux: sample actual CPU time, including time in native code
java -
XX:StartFlightRecording:filename=cpu.jfr,jdk.CPUTimeSample#enabled=true,jdk.CPUTimeSample#throttle=10ms -jar app.jar
jfr summary cpu.jfr | grep -E "CPUTimeSample|ExecutionSample"
jfr print --events jdk.CPUTimeSample cpu.jfr | head -40

# New event types for method-level timing and tracing (JEP 520)
jfr summary cpu.jfr | grep -E "MethodTiming|MethodTrace"
```

💡 **Pro tip**

JEP 536 (JDK 27) addresses a practical worry: recordings may contain secrets (command-line arguments, system properties, environment values). Redacting them inside the JVM before the file is written makes recordings safer to hand to a vendor or attach to a ticket.

Section 03

Runtime, GC & Start-up, Java 24–27

Faster start-up with the AOT cache, smaller objects with compact headers, G1 everywhere, and the 'integrity by default' changes that cause upgrade warnings.

Questions 413–418 · 6 questions



🏁 **Q413** MJ-074 • Advanced

What is the AOT cache (Project Leyden), and how do you use it?

⚡ **Quick answer**

The AOT cache stores the result of a *training run* (loaded and linked classes, method profiles and, since 26, heap objects for any GC) in a file the JVM reuses at start-up. Java 24 introduced AOT class loading and linking (JEP 483), 25 made it one command and added method profiles (JEP 514, 515), and 26 made object caching work with any GC (JEP 516).

A rough measurement from our sandbox, JDK 25, a trivial hello-world app, average of 5 runs:

© TechTribers. For personal use only. Do not redistribute.

Mode	Start-up
No class-data sharing	~120 ms
Default CDS (JDK classes)	~54 ms
AOT cache	~35 ms

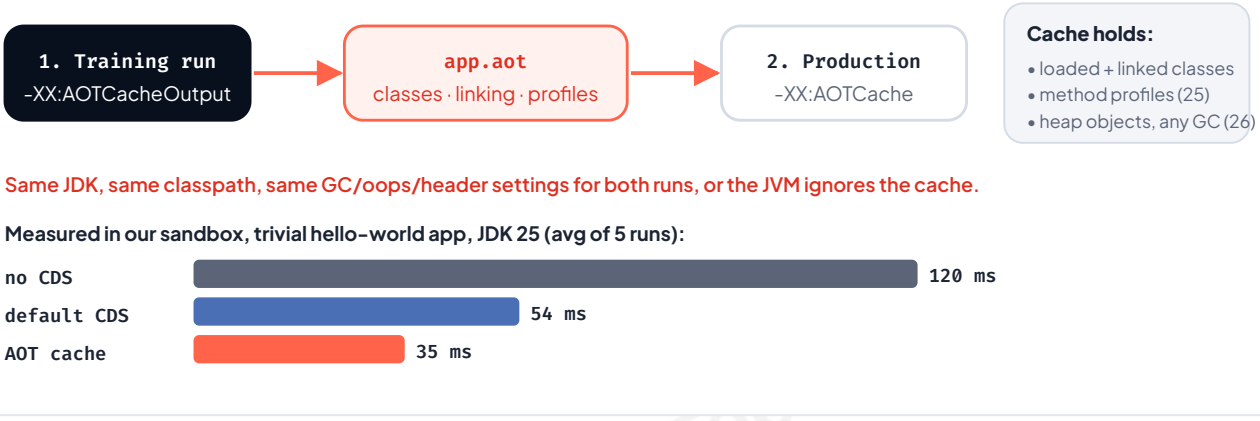
A real Spring Boot service loads thousands of classes, so the saving is larger in absolute terms. Measure your own app: the cache is only useful if the training run exercises the same code as production.

Terminal

```
# 1. Training run: execute a typical workload, then let the app exit. One command in JDK 25+ (JEP 514)
java -XX:AOTCacheOutput=app.aot -cp app.jar com.tt.Main

# 2. Production run: reuse the cache
java -XX:AOTCache=app.aot -cp app.jar com.tt.Main

# JDK 26+: the cache works with any collector, e.g. ZGC (JEP 516)
java -XX:AOTCacheOutput=app.aot -XX:+UseZGC -cp app.jar com.tt.Main
```



⚠️ **Interview trap**

The cache records the JDK version, classpath and options (GC, compressed oops, object-header mode) it was built with. If production differs, the JVM silently ignores it. Build the cache in CI from the same image you deploy.

© TechTribers. For personal use only. Do not redistribute.

Q414 MJ-075 • Advanced

Compact headers are the default in JDK 27: how much heap do they really save, and how do you roll them out?

⚡ **Quick answer**

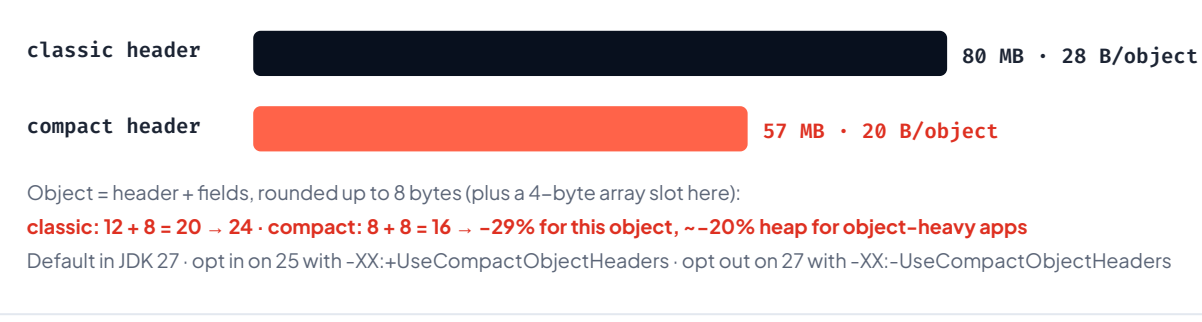
On JDK 27 you get them without doing anything (JEP 534); on JDK 25 they are a product flag (JEP 519, experimental in 24). In our measurement 3 million small records dropped from 80 MB to 57 MB. Roll them out by comparing heap and GC logs before and after, and keep `-XX:-UseCompactObjectHeaders` as the escape hatch.

The arithmetic: a classic `P(int, int)` object is 12 (header) + 8 (fields) = 20 bytes, rounded up to 24. With a compact header it is 8 + 8 = 16, a one-third cut for this object. Plus a 4-byte reference slot in the array gives the 28 vs 20 bytes per element measured above.

Real applications gain less because strings, arrays and primitives dominate; OpenJDK's own estimates put heap savings around 10–20% for object-heavy workloads, with lower GC pressure and better cache behaviour as a side effect.

Terminal

```
# Heap used by 3,000,000 live records, each: record P(int x, int y)  (measured, JDK 25 and 27)
java -XX:+UseSerialGC -XX:-UseCompactObjectHeaders Heap     # JDK 27, classic:  80 MB  (28 bytes per object incl. array slot)
java -XX:+UseSerialGC -XX:+UseCompactObjectHeaders Heap     # JDK 27, compact:   57 MB  (20 bytes per object incl. array slot)
java -XX:+UseSerialGC Heap                                   # JDK 25 default:    80 MB  (compact must be switched on)
java -XX:+UseSerialGC -XX:+UseCompactObjectHeaders Heap     # JDK 25 + flag:     57 MB
```



⚠️ **Interview trap**

Nothing to change in code, but check anything that depends on exact object sizes or header layout (some off-heap or serialization tricks, and agents that read headers). Opt out on JDK 27 with `-XX:-UseCompactObjectHeaders` if you hit a problem.

© TechTribers. For personal use only. Do not redistribute.

Q415 MJ-076 • Advanced

What changed for garbage collectors between Java 22 and 27, and why is G1 now the default everywhere?

⚡ **Quick answer**

Highlights: generational ZGC became the default ZGC mode (23) and non-generational ZGC was removed (24); Shenandoah went generational (25); G1 got region pinning (22) and lower synchronisation overhead (26). JEP 523 (JDK 27) makes **G1 the default in all environments**; before, JVMs on a machine with under 2 CPUs or ~1.8 GB of memory silently picked the Serial collector.

Release	GC change
22	G1 region pinning for JNI critical sections (JEP 423)
23	ZGC defaults to generational mode (JEP 474)
24	Non-generational ZGC removed (JEP 490); generational Shenandoah (experimental)
25	Generational Shenandoah becomes a product feature (JEP 521)
26	G1 throughput improved by reducing synchronisation (JEP 522)
27	G1 is the default in all environments (JEP 523)

Terminal

```
# Which collector does the JVM pick? Measured with -XX:+PrintFlagsFinal
java -XX:ActiveProcessorCount=1 -Xmx512m -XX:+PrintFlagsFinal -version | grep -E " UseSerialGC | UseG1GC "
#   JDK 21:  UseG1GC = false   UseSerialGC = true     <- small container
#   JDK 25:  UseG1GC = false   UseSerialGC = true
#   JDK 27:  UseG1GC = true    UseSerialGC = false    <- JEP 523

java -XX:ActiveProcessorCount=4 -Xmx4g -XX:+PrintFlagsFinal -version | grep -E " UseSerialGC | UseG1GC "
#   JDK 21 / 25 / 27:  UseG1GC = true
```

	JDK 21	JDK 25	JDK 27
≥ 2 CPUs and ≥ ~1.8 GB	G1	G1	G1
1 CPU or small memory (e.g. 1 CPU / 512 MB)	Serial	Serial	G1

Results printed by -XX:+PrintFlagsFinal with -XX:ActiveProcessorCount and -Xmx set (JEP 523 in JDK 27).

Why it matters: pods limited to 1 CPU used to get the single-threaded Serial collector silently.

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> If a service moved to JDK 27 in a 1-CPU, 512 MB pod, its collector changes from Serial to G1 with no flag change, so re-check memory overhead and pause times after upgrading.

🔒 **Q416** `MJ-077` • Advanced

What is "integrity by default", and which changes trigger new warnings after an upgrade?

> ⚡ **Quick answer**
> A set of JDK changes making sure code can't silently undermine the runtime: the Security Manager is permanently disabled (JEP 486, Java 24), JNI and native access need explicit permission (JEP 472), `sun.misc.Unsafe` memory methods are deprecated and now warn (JEP 471, 498), dynamic agent loading warns (JEP 451), and mutating `final` fields reflectively is being locked down (JEP 500).

Warning you may see	Cause	Fix
Native access warning	JNI / FFM without permission	`--enable-native-access=ALL-UNNAMED` (or the module name)
`sun.misc.Unsafe` **memory-access warning**	Old libraries (Netty, Kryo, Hadoop…)	Upgrade the library; replace with `VarHandle` / FFM
Dynamic agent loaded	Mockito inline, profilers attaching later	Use `-javaagent` at launch, or opt in with `-XX:+EnableDynamicAgentLoading`
Final field mutated	Reflective `Field.set` on a `final` (Java 26+)	Upgrade the library; temporary `--enable-final-field-mutation`
`UnsupportedOperationException` **on** `setSecurityManager`	Legacy sandboxing code	Use OS/container isolation instead

Java

```
try { System.setSecurityManager(new SecurityManager()); System.out.println("installed"); }
catch (UnsupportedOperationException e) { System.out.println("UnsupportedOperationException: " +
e.getMessage()); }
```

▶ **Output**

```
UnsupportedOperationException: Setting a Security Manager is not supported
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> The principle to state in an interview: the JDK is moving from "everything is allowed unless blocked" to "powerful operations need explicit opt-in", which improves security and lets the JIT make stronger assumptions (a base for features like `final` constant folding).

Q417 MJ-078 • Advanced

What does JEP 500, "Prepare to Make Final Mean Final", change?

> ⚡ **Quick answer**
> Reflection can modify `final` instance fields via `Field.set` after `setAccessible(true)`. Java 26 starts warning when that happens, and a future release will block it unless explicitly enabled. Allowing it prevents the JIT from treating `final` fields as true constants.

Measured behaviour of the same program: on JDK 25 it silently mutates; on JDK 26 and 27 it still mutates but prints a warning ("Final field region ... has been mutated reflectively ... Mutating final fields will be blocked in a future release unless final field mutation is enabled"); with `--illegal-final-field-mutation=deny` it throws `IllegalAccessException` as above.

Options: `--illegal-final-field-mutation=allow|warn|debug|deny`, and `--enable-final-field-mutation=ALL-UNNAMED` (or a module name) to permit it deliberately without warnings.

Java

```
static class Config { private final String region = "ap-south-1"; }

Config c = new Config();
Field f = Config.class.getDeclaredField("region");
f.setAccessible(true);
try { f.set(c, "eu-west-1"); System.out.println("mutated"); }
catch (IllegalAccessException e) { System.out.println("denied: " + e.getClass().getSimpleName()); }
```

▶ Output

```
denied: IllegalAccessException
```



`--illegal-final-field-mutation=allow | warn | debug | deny --enable-final-field-mutation=MODULE (opt in)`

Why: the JIT can only trust and constant-fold final fields if they truly never change (JEP 500, 'integrity by default').

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Libraries that inject values into `final` fields (some serialization, mocking and DI tools) will start printing warnings after you upgrade. Update them before turning warnings into failures with `deny`.

🗑️ **Q418** `MJ-079` • Intermediate

What was removed or disabled between Java 22 and 27?

⚡ **Quick answer**
The Applet API is gone (26), the Security Manager can no longer be enabled (24), 32-bit x86 ports were removed (24–25), non-generational ZGC was removed (24), and several risky legacy behaviours now warn or require opt-in.

Each removal was announced years earlier:

- **Applet API**: deprecated in Java 9, marked for removal in 17, removed in 26 (JEP 504). The browsers dropped plug-in support long ago.
- **Security Manager**: deprecated for removal in 17; permanently disabled in 24 (JEP 486).
- **32-bit x86**: the Windows port was removed in 24 (JEP 479) and the remaining 32-bit x86 ports in 25 (JEP 503).

Terminal

```
$ javac Ap.java     # class Ap extends java.applet.Applet
# JDK 25:  Ap.java:2: warning: [removal] Applet in java.applet has been deprecated and marked for removal
# JDK 26:  Ap.java:1: error: package java.applet does not exist
```

Applet API	26	JEP 504 · package java.applet no longer exists
Security Manager	24	JEP 486 · setSecurityManager throws UnsupportedOperationException
32-bit x86 ports	24–25	JEP 479 (Windows) and JEP 503 (all) · 64-bit only
Non-generational ZGC	24	JEP 490 · ZGC is generational only
Unsafe memory access	23→	JEP 471 deprecated, JEP 498 warns at runtime · use VarHandle / FFM
Dynamic agents	21→	JEP 451 warns today · opt in with -XX:+EnableDynamicAgentLoading

💡 **Pro tip**
Use `jdeprscan --release 25 app.jar` before an upgrade to find use of APIs that are deprecated for removal. It's much cheaper than discovering them from a failed build.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Previews & Incubators in JDK 27

Features you can try with --enable-preview today, and the Project Valhalla changes arriving next.

Questions 419–422 · 4 questions



Q419 MJ-080

• Advanced

What are primitive types in patterns, `instanceof` and `switch`?

⚡ Quick answer

This preview (fifth preview in JDK 27, JEP 532; first previewed in Java 23) lets patterns, `instanceof` and `switch` work with **primitive types**, not only reference types. A primitive pattern matches only when the conversion is exact and lossless, such as `42 instanceof byte b` (true) or `1000 instanceof byte b` (false).

Why it exists: removing the special-casing of primitives makes pattern matching uniform, avoids hand-written range checks, and lets records with primitive components be deconstructed with primitive patterns (`Num(double v)`).

Java

```
record Num(double v) { }
static String bucket(int i) {
    return switch (i) {
        case 0 -> "zero";
        case int n when n < 0 -> "negative";
        case int n when n > 100 -> "big";
        case int n -> "small " + n;
    };
}
static String longKind(long l) { return switch (l) { case 1L -> "one"; case long x when x >
1_000_000L -> "million+"; case long x -> "other"; }; }
static String viaBoolean(boolean b) { return switch (b) { case true -> "yes"; case false -> "no"; };
}

int i = 42;
System.out.println((i instanceof byte b) + " " + (1000 instanceof byte c) + " " + (i instanceof
short s ? "fits short " + s : "no"));
System.out.println(bucket(0) + " " + bucket(-5) + " " + bucket(500) + " " + bucket(7));
System.out.println(longKind(1L) + " " + longKind(5_000_000L) + " " + longKind(9L));
System.out.println(viaBoolean(true) + " " + viaBoolean(false));
Object o = new Num(3.0);
if (o instanceof Num(double v) && v > 2) System.out.println("record with primitive pattern: " + v);
double d = 3.0; float f = 3.5f;
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println((d instanceof int n ? "exact int " + n : "not exact") + " " + (f instanceof int m
? "exact int " + m : "not exact"));
```

▶ Output

```
true false fits short 42
zero negative big small 7
one million+ other
yes no
record with primitive pattern: 3.0
exact int 3 not exact
```

A primitive pattern matches only when the conversion is lossless (JDK 27: fifth preview)

`42 instanceof byte b`	`true`	fits in a byte
`1000 instanceof byte b`	`false`	1000 > 127: would lose data
`3.0 instanceof int n`	`true`	exact integer
`3.5f instanceof int n`	`false`	fraction would be lost
`switch (int i) { case int n when n > 100 → … }`	`ok`	patterns and guards on int, long, boolean…

⚠️ **Interview trap**

Preview feature: compile and run with `--enable-preview`, on the exact JDK version. The language team has adjusted details between previews (for example tighter dominance rules in JDK 26).

💤 **Q420** MJ-081 • Advanced

What are Lazy Constants (formerly Stable Values)?

⚡ **Quick answer**

A `LazyConstant<T>` holds a value computed **on first use** and then fixed forever, giving the JIT the same trust as a `final` field without the cost of eager initialisation. It started as `StableValue` (preview in Java 25), was renamed `LazyConstant` in 26, and is in its third preview in JDK 27 (JEP 531).

Compared with the alternatives:

- `static final` field: computed during class initialisation, so expensive values slow start-up even when never used.
- Double-checked locking / holder idiom: boilerplate and easy to get wrong; the JIT can't treat the field as a constant.
- `LazyConstant`: thread-safe, computed at most once, and optimised like a final field after initialisation.

JDK 27's third preview removed `isInitialized()` and `orElse()`, and added lazy collection factories: `List.ofLazy(size, mapper)`, `Set.ofLazy(candidates, predicate)` and `Map.ofLazy(keys, mapper)`, where each

© TechTribers. For personal use only. Do not redistribute.

element is computed on first access.

```java
static final LazyConstant<String> DB_URL = LazyConstant.of(() -> {
    System.out.println("computing DB_URL once");
    return "jdbc:postgresql://db/tt";
});
static final List<Integer> SQUARES = List.ofLazy(5, i -> { System.out.println("square " + i); return
i * i; });

System.out.println("start");
System.out.println(DB_URL.get());
System.out.println(DB_URL.get());                    // no recomputation
System.out.println(SQUARES.get(3) + " " + SQUARES.get(3) + " " + SQUARES.size());
```

▶ Output

```
start
computing DB_URL once
jdbc:postgresql://db/tt
jdbc:postgresql://db/tt
square 3
9 9 5
```



💡 **Pro tip**

Good candidates: loggers, parsed configuration, compiled regex patterns, expensive lookup tables and singletons that are costly to build.

📋 **Q421** MJ-082 • Intermediate

Which features are still preview or incubating in JDK 27, and when will they be final?

⚡ **Quick answer**

As of JDK 27: Structured Concurrency (seventh preview), Lazy Constants (third), Primitive Types in Patterns (fifth), PEM Encodings (third, final planned for JDK 28 as JEP 542), and the Vector API (twelfth incubator). Everything else from Java 22–25 discussed in this part is already final.

Interview-ready facts:

© TechTribers. For personal use only. Do not redistribute.

- A preview can change between releases. Example verified for this book: structured concurrency's `Joiner.anySuccessfulResultOrThrow()` (JDK 25) became `anySuccessfulOrThrow()` in JDK 27, and its default failure type changed from `FailedException` to a checked `ExecutionException`.
- Class files compiled with preview features are marked and **only run on the same JDK feature release**.
- The Vector API has been incubating since Java 16 because it waits for Project Valhalla's value types (next question).

Terminal

```
# Compile and run preview features (the JDK version must match exactly)
javac --release 27 --enable-preview Main.java
java --enable-preview Main

# Incubator modules are separate: add them explicitly
java --add-modules jdk.incubator.vector Main
```

Feature	Stage in JDK 27	JEP	Outlook
Structured Concurrency	7th preview	JEP 533	API still changing
Lazy Constants	3rd preview	JEP 531	next: 4th preview (draft)
Primitive types in patterns	5th preview	JEP 532	close to final
PEM encodings	3rd preview	JEP 538	final planned in JDK 28 (JEP 542)
Vector API	12th incubator	JEP 537	waits for Valhalla

Everything here needs --enable-preview (or --add-modules for incubators) and pins you to one exact JDK version.

⚠️ **Interview trap**

Don't put preview APIs in library code or production services. Use them in experiments and learn them for interviews, then adopt each one after it becomes final.

💎 **Q422** MJ-083 • Advanced

What is Project Valhalla, and what is coming in JDK 28?

⚡ **Quick answer**

Valhalla adds **value classes and objects**: objects without identity whose equality is by field values, so the JVM can store them flattened inside arrays and other objects with no header or pointer. JEP 401 (Value Classes and Objects) and JEP 539 (Strict Field Initialization in the JVM) are targeted for **preview in JDK 28** (March 2027).

Why it matters:

- **Performance**: an array of 1 million `Point` values becomes one contiguous block instead of 1 million pointers to separate objects, with better cache locality and less GC work.

© TechTribers. For personal use only. Do not redistribute.

- **Fewer wrapper costs**: types such as `Integer` and `LocalDate` can become value classes, reducing allocation and `==` surprises.
- **Unblocks the Vector API**, which is waiting for value types before leaving incubation.

Timeline as of October 2026: JDK 28 feature freeze in early December 2026, general availability March 2027. JEP 542 (PEM encodings final) is also planned for 28.

```java
// Illustrative only: JEP 401 syntax is still in preview and may change
value class Point {
    private final int x, y;                // fields are implicitly final
    Point(int x, int y) { this.x = x; this.y = y; }
}

Point[] grid = new Point[1_000_000];       // may be stored flattened: x,y,x,y,... with no per-element object
```

Identity objects today vs value objects (JEP 401, targeted for a JDK 28 preview)



Strict field initialisation (JEP 539) lets the JVM guarantee fields are set before use. Both arrive as preview in JDK 28 (March 2027).

⚠️ **Interview trap**

Until it ships, don't design APIs around Valhalla's details. A good interview answer explains the *goal* (identity-free, flattened data) and acknowledges the syntax and rules are still in preview.

© TechTribers. For personal use only. Do not redistribute.

Section 05

Java 25 LTS, 26 & 27, and Choosing a Version

What each recent release contains, how to upgrade from 21, which version to run in production, and a ready-made 60-second answer.

Questions 423–427 · 5 questions



Q423 MJ-084

• Intermediate

What's new in Java 25, the current LTS?

⚡ Quick answer

Java 25 (September 2025) is an LTS with 18 JEPs. Final features: scoped values, flexible constructor bodies, module imports, compact source files, the KDF API, compact object headers (opt-in), AOT ergonomics and method profiling, generational Shenandoah, and JFR enhancements. Previews include structured concurrency and primitive patterns.

Area	JEP	Feature
Language	506	Scoped Values (final)
	513	Flexible Constructor Bodies (final)
	511 / 512	Module Import Declarations · Compact Source Files and Instance Main Methods (final)
	507	Primitive Types in Patterns (third preview)
Libraries	510	Key Derivation Function API (final)
	505 / 502	Structured Concurrency (fifth preview) · Stable Values (preview)
	470 / 508	PEM Encodings (preview) · Vector API (tenth incubator)
Runtime	519	Compact Object Headers (product, opt-in)
	514 / 515	AOT Command-Line Ergonomics · AOT Method Profiling
	521	Generational Shenandoah (product)
Tooling	509 / 518 / 520	JFR CPU-Time Profiling (experimental) · Cooperative Sampling · Method Timing and Tracing
Platform	503	Remove the 32-bit x86 Port

© TechTribers. For personal use only. Do not redistribute.

P = preview · E = experimental · F = final (F* = product, opt-in flag) · D = on by default

	Java 22	Java 23	Java 24	Java 25	Java 26	Java 27
Unnamed variables _	F					
FFM API	F					
Stream Gatherers	P	P	F			
Class-File API	P	P	F			
Scoped Values	P	P	P	F		
Flexible constructors	P	P	P	F		
Compact source files	P	P	P	F		
Compact object headers			E	F*		D
HTTP/3 client					F	
Primitive patterns		P	P	P	P	P
Structured concurrency	P	P	P	P	P	P

💡 **Pro tip**

Java 25 also contains everything from 22, 23 and 24 that went final in those releases: unnamed variables, FFM, stream gatherers, the Class-File API, virtual threads without `synchronized` pinning and the first post-quantum algorithms.

Q424 MJ-085 • Intermediate

What did Java 26 and Java 27 add?

⚡ **Quick answer**

Both are non-LTS releases with 10 and 9 JEPs. Java 26 (March 2026): HTTP/3, AOT object caching with any GC, the Applet API removal, G1 throughput, and "final means final" warnings. Java 27 (September 2026): G1 default everywhere, compact object headers on by default, hybrid post-quantum TLS and JFR in-process redaction.

Release	Final features	Preview / incubator
26 (Mar 2026)	500 Prepare to Make Final Mean Final · 504 Remove the Applet API · 516 AOT Object Caching with Any GC · 517 HTTP/3 for the HTTP Client · 522 G1 throughput	524 PEM (2nd) · 525 Structured Concurrency (6th) · 526 Lazy Constants (2nd) · 530 Primitive Patterns (4th) · 529 Vector API (11th)
27 (Sep 2026)	523 G1 as default everywhere · 527 Post-Quantum Hybrid Key Exchange for TLS 1.3 · 534 Compact Object Headers by Default · 536 JFR In-Process Data Redaction	531 Lazy Constants (3rd) · 532 Primitive Patterns (5th) · 533 Structured Concurrency (7th) · 538 PEM (3rd) · 537 Vector API (12th)

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Five of the nine JEPs in JDK 27 are still previews or incubators, which is why it was called a quiet release: it is mostly about making earlier work default and refining what's coming.

⬆️ **Q425** `MJ-086` • Advanced

How do you upgrade a service from Java 21 to 25, and what are the gains?

> ⚡ **Quick answer**
> Upgrade the toolchain and bytecode libraries first (class-file version 69), run the test suite and a load test on 25, deal with new warnings (native access, `Unsafe`, agents), then adopt opt-in features one at a time: compact object headers, the AOT cache and scoped values. The jump is low-risk for most services.

Checklist

- Build: Maven/Gradle toolchain 25, and versions of ASM, Byte Buddy, Mockito, Jacoco, Lombok and Spring that understand Java 25 class files.
- Find problems early: `jdeps --jdk-internals`, `jdeprscan --release 25`, and run tests with `-Xlog:disable -Xlog:all=warning` to see JDK warnings.
- Remove Security Manager usage. `System.setSecurityManager` now throws `UnsupportedOperationException`.
- Native code and old libraries may trigger JNI or `Unsafe` warnings. Upgrade the library, or add `--enable-native-access` deliberately.
- 32-bit and Windows-x86 specific code is gone.

What you gain on 25

- Runtime: opt-in compact object headers (about 20% less heap for object-heavy apps), a one-command AOT cache for faster start-up and warm-up, generational Shenandoah if you use it.
- Language and libraries: scoped values, flexible constructors, stream gatherers, the Class-File API, the KDF API, ML-KEM and ML-DSA.
- Virtual threads no longer pin carriers inside `synchronized` (JEP 491, Java 24), which removes the most common adoption caveat from Java 21.

> 💡 **Pro tip**
> Upgrade in two steps in CI: first run the existing test suite on 25 with no flags, then enable one opt-in optimisation per change so any regression has an obvious cause.

© TechTribers. For personal use only. Do not redistribute.

Q426 MJ-087 • Intermediate

Which Java version should you run in production in 2026?

⚡ **Quick answer**

Run an **LTS**: Java 25 for new work, Java 21 if you're mid-migration. Non-LTS releases (26, 27) are production-quality but receive updates only until the next release, so use them for early validation, not as the long-term baseline. The next LTS is Java 29 (September 2027).

Version	Type	Use it for
17	LTS (2021)	Existing services; plan the move to 21 or 25
21	LTS (2023)	Stable baseline with virtual threads and pattern matching
25	LTS (2025)	Default for new services: scoped values, compact headers, AOT cache
26 / 27	Non-LTS	Trying features early, validating upcoming changes in CI
29	Next LTS (Sep 2027)	Plan the next upgrade cycle

Also check your frameworks' support matrix (Spring Boot, Hibernate, Mockito, your cloud runtime). They usually support a new LTS within a few months of release, and that, more than the JDK itself, often sets the real upgrade date.



LTS every two years (orange). Non-LTS releases are production-quality but supported only until the next release.

⚠️ **Interview trap**

"Always upgrade to the latest" and "never upgrade until forced" are both weak answers. A good one explains a cadence: LTS to LTS, with a CI job that builds against the latest release so surprises show up early.

© TechTribers. For personal use only. Do not redistribute.

🎤 Q427 MJ-088 • Intermediate

"What has changed in Java since Java 8?" Give a strong 60-second answer.

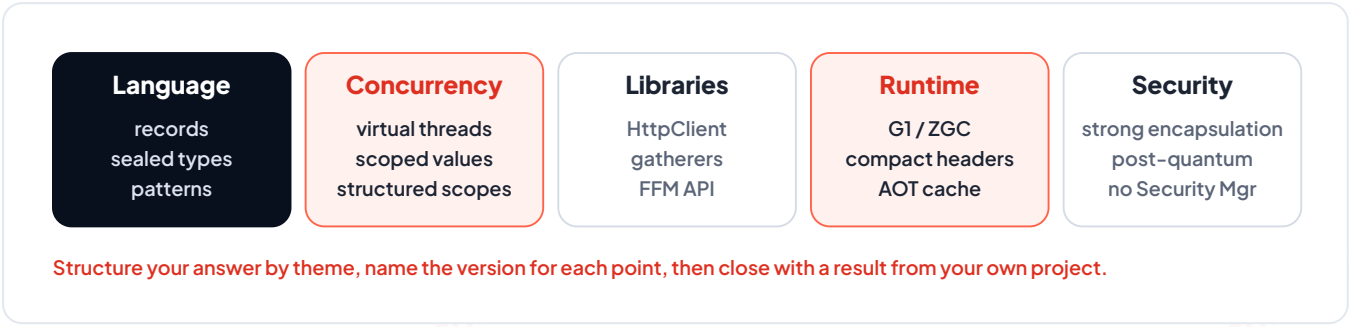
⚡ Quick answer

Group the answer by theme and name a version for each point: language (records, sealed types, pattern matching, text blocks), concurrency (virtual threads, scoped values), libraries (HttpClient, gatherers, FFM), runtime (G1/ZGC, compact headers, AOT cache) and security (strong encapsulation, post-quantum crypto).

A sample answer, about 60 seconds:

- **Language**: from `var` in 10 to text blocks and switch expressions (14–15), records (16), sealed classes (17), pattern matching for `switch` and record patterns (21), and unnamed variables (22). Data-oriented code is now much shorter and the compiler checks it exhaustively.
- **Concurrency**: virtual threads in 21 let plain blocking code scale to very high concurrency; scoped values (25) replace `ThreadLocal`; structured concurrency is next, still in preview.
- **Libraries**: the built-in `HttpClient` (11, with HTTP/3 in 26), stream gatherers (24), the FFM API (22) replacing JNI, and the Class-File API (24).
- **Runtime**: G1 default since 9 and everywhere in 27, generational ZGC, compact object headers (default in 27) and an AOT cache for start-up (24–26), plus container awareness.
- **Platform and security**: modules, strong encapsulation of JDK internals, the Security Manager gone, post-quantum crypto (24, 27), and a six-month release cadence with LTS every two years.

"In my last project we moved from 8 to 17 and then 21: records and sealed types removed a lot of boilerplate, and virtual threads let us drop a custom thread-pool layer."



💡 Pro tip

Close with a concrete result from your own work. Interviewers remember an upgrade that cut memory, removed a library or fixed an incident far longer than a list of JEP numbers.

© TechTribers. For personal use only. Do not redistribute.

Series progress

427 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 11 starts at Q428.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.